

TEORI BAHASA DAN AUTOMATA

Modul Pembelajaran: Deterministic Finite Automata (DFA)

Penjelasan mendalam mengenai konsep dasar, cara kerja, struktur loop transisi, perhitungan matematis, serta studi kasus perancangan mesin DFA.

Contents

1	Mengenal Deterministic Finite Automata (DFA)	2
1.1	Apa itu Mesin DFA?	2
1.2	Anatomi Diagram Transisi DFA	2
2	Loop Transisi dan Dead State	3
2.1	Kenapa Harus Ada Loop Transisi?	3
2.2	Trap State (Dead State)	3
3	Definisi Formal dan Hitungan Matematis	4
3.1	Merangkum ke Tabel Transisi	4
3.2	Fungsi Transisi Diperluas (δ^*)	4
4	Kumpulan Studi Kasus Perancangan DFA	5
4.1	Kasus 1: Mesin Pengecek Tepat Pola "abba"	5
4.2	Kasus 2: Minimal Ada Satu Angka "1"	5
4.3	Kasus 3: Harus Diawali Angka "0"	6
4.4	Kasus 4: Panjang String Pas Banget 2 Karakter	6
4.5	Kasus 5: Pengecekan Pola Terstruktur ($a^n b$)	6
4.6	Kasus 6: Tracking Panjang Fungsi Transisi (δ^*)	7

1 Mengenai Deterministic Finite Automata (DFA)

1.1 Apa itu Mesin DFA?

Bayangin *Deterministic Finite Automaton* (DFA) itu kayak sebuah mesin pemindai otomatis yang tugasnya ngebaca rentetan teks atau karakter. Mesin ini dikasih sebuah pita input (*Input Tape*) yang isinya deretan karakter (kita sebut *string*), dan mesin bakal ngebaca pita itu satu per satu dari arah kiri ke kanan.

Selesai ngebaca semua teks di pita, mesin ini cuma punya dua pilihan jawaban akhir: teksnya diterima (**Accept**) atau teksnya ditolak (**Reject**). Nggak ada jawaban "mungkin" atau "setengah diterima".

Sifat paling utama dari DFA adalah **Determinisme**. Maksudnya, kalau mesin lagi ada di suatu *state* (posisi/kedudukan tertentu) dan dia ngebaca sebuah karakter, langkah atau arah jalan mesin selanjutnya udah pasti dan nggak bisa ditawar.

- Nggak ada cerita satu input bikin mesin bingung milih dua jalan. Tiap input pasti mengarah ke tepat 1 *state* tujuan.
- Setiap *state* wajib banget punya jalur panah keluar untuk **semua** simbol yang ada di kamus inputnya.

1.2 Anatomi Diagram Transisi DFA

Biar lebih gampang dimengerti, alur kerja DFA biasanya digambar pakai diagram. Di dalam diagram ini ada beberapa bagian penting:

- **Initial State:** Titik mulai (*start*) jalannya mesin. Biasanya ditandai pakai panah yang masuk tanpa ada asal usul statenya.
- **State:** Posisi atau memori mesin pada saat itu, digambar pakai lingkaran biasa.
- **Transition:** Garis panah yang nunjukin perpindahan dari satu *state* ke *state* lain.
- **Input:** Karakter (angka/huruf) yang ditulis di atas garis transisi. Ini yang jadi pemicu mesin buat pindah.
- **Final State:** *State* penentu akhir yang digambar pakai lingkaran ganda (*double circle*). Mesin DFA boleh banget punya lebih dari satu *final state* di dalam satu diagram.

Nah, syarat *string* dibilang **diterima** itu lumayan ketat. Mesin harus udah selesai baca **semua** karakter sampai ujung pita, dan pas karakter terakhir dibaca, posisi mesin harus berhenti pas di salah satu *Final State*. Kalau berhentinya di *state* biasa, ya berarti *string*-nya **ditolak**.

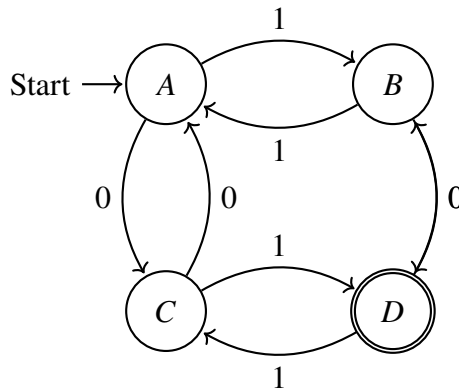


Figure 1: Ilustrasi Struktur Diagram Transisi DFA. Lingkaran ganda D adalah Final State.

2 Loop Transisi dan Dead State

2.1 Kenapa Harus Ada Loop Transisi?

Pas bikin DFA, kita bakal sering banget bikin panah transisi yang muter balik ke *state* itu sendiri. Ini yang disebut dengan **Loop**. Artinya, mesin baca karakter baru, tapi posisinya nggak pindah ke *state* lain. Secara matematis ditulis $\delta(q_i, a) = q_i$.

Loop punya tiga peran krusial di DFA:

1. **Menjaga Kondisi Logika:** Misal kita disuruh nyari pola "minimal ada angka 1". Kalau angka 1 udah ketemu, kita nggak mau mesinnya ngelupain fakta itu kalau dikasih angka 0 lagi. Loop ngebantu mesin buat tetep "ingat" dan stabil di kondisi tersebut.
2. **Bikin Desain Efisien:** Daripada bikin *state* baru yang fungsinya sama aja, mending pakai loop. Contoh, buat ngecek jumlah angka 0 itu genap atau ganjil, angka "1" kan nggak ngerubah hitungan genap-ganjilnya angka 0. Jadi tiap ketemu "1", kita loop aja di *state* yang sama.
3. **Syarat Determinisme:** Balik lagi ke aturan awal, setiap *state* **harus** punya transisi buat semua simbol. Kalau ada simbol yang nggak bikin pindah logika, ya kasih loop aja biar syaratnya terpenuhi dan mesin nggak *error* (ngehang).

2.2 Trap State (Dead State)

Ada satu tipe *state* yang unik banget, namanya **Trap State** atau **Dead State**. Ini adalah *state* "lubang hitam" yang isinya cuma loop untuk semua simbol yang ada.

- Kalau di rumus ditulis: $\delta(q_t, 0) = q_t$ dan $\delta(q_t, 1) = q_t$.
- Begitu mesin masuk ke *state* ini, dia terperangkap dan nggak bakal bisa keluar lagi ke *state* mana pun.
- Gunanya murni cuma buat "menyerap" semua sisa karakter di pita kalau mesin udah tahu dari awal string-nya pasti bakal ditolak.
- Karena *trap state* itu **bukan final state**, string apa pun yang lewat ke sini dipastikan 100% **ditolak**.

3 Definisi Formal dan Hitungan Matematis

Biar nggak cuma nggambar doang, mesin DFA ini di-modelin pakai matematika menjadi lima komponen utama (5-tupel): $M = (Q, \Sigma, \delta, q_0, F)$. Penjelasan rincinya begini:

- **Q (Himpunan State):** Ini adalah list semua *state* yang ada di dalam mesin. Misalnya $Q = \{q_0, q_1, q_2, q_3\}$.
- **Σ (Input Alphabet):** Kamus karakter yang bisa dibaca sama mesin. Ingat, *empty string* (ϵ) nggak boleh masuk ke himpunan ini. Contohnya $\Sigma = \{a, b\}$.
- **δ (Fungsi Transisi):** Rumus aturannya. Dipetakan sebagai $\delta : Q \times \Sigma \rightarrow Q$. Fungsinya buat nentuin: "kalau aku di state ini, dan dapet huruf ini, aku harus lompat ke state mana?".
- **q_0 (State Awal):** Tempat komputasi dimulai. Wajib hukumnya q_0 ada di dalam himpunan Q .
- **F (Himpunan Final State):** Kumpulan *state* penerima. Karena final state itu ya bagian dari state biasa, maka $F \subseteq Q$.

3.1 Merangkum ke Tabel Transisi

Biar fungsi δ gampang dibaca, kita biasa bikin jadi tabel. Baris tabel buat *state* asal, dan kolom tabel buat input simbolnya. Isian tabelnya adalah *state* tujuannya.

δ	a	b
$\rightarrow *q_0$	q_1	q_5
$*q_1$	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
$*q_4$	q_5	q_5
q_5	q_5	q_5

Table 1: Contoh Tabel Transisi DFA. Tanda ($\rightarrow$) nandain state awal, tanda (*) nandain final state. State q_5 terlihat sebagai Dead State karena selalu balik ke dirinya sendiri.

3.2 Fungsi Transisi Diperluas (δ^*)

Fungsi δ itu kan cuma ngebaca satu karakter. Nah, gimana caranya ngitung kalau inputnya satu kalimat utuh (string)? Kita pakai **fungsi transisi diperluas** (δ^*). Formatnya: $\delta^* : Q \times \Sigma^* \rightarrow Q$. Ini bacanya: $\delta^*(q, w) = q'$ yang berarti dari state q , kalau dikasih deretan string w , dia bakal jalan nelusurin transisi satu-satu sampai ujungnya berhenti di state q' .

Berdasarkan ini, kita bisa ngerumusin **Bahasa yang Diterima** ($L(M)$) secara matematis:

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$$

Artinya: Himpunan semua string w yang kalau dijalankan dari state awal (q_0) bakal mendarat cantik di dalam himpunan *final state* (F).

4 Kumpulan Studi Kasus Perancangan DFA

Di bagian ini kita bakal bedah tuntas berbagai contoh kasus dari pola bahasa yang sederhana sampai yang butuh analisis panjang.

4.1 Kasus 1: Mesin Pengecek Tepat Pola "abba"

Mesin ini didesain spesifik cuma nerima satu buah string aja, yaitu $L = \{abba\}$. Selain itu pasti gagal.

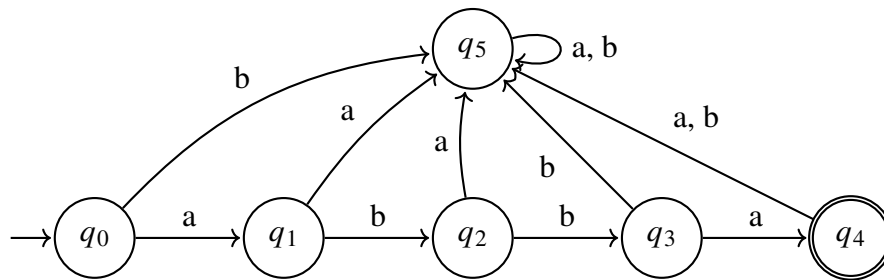


Figure 2: DFA Penerima "abba". q_5 adalah Trap State.

Gimana jalannya kalau kita masukin string yang bener ($abba$)? 1. Mulai di q_0 , dapet 'a' → masuk ke q_1 . 2. Di q_1 , dapet 'b' → masuk ke q_2 . 3. Di q_2 , dapet 'b' → masuk ke q_3 . 4. Di q_3 , dapet 'a' → masuk ke q_4 . Karena udah habis stringnya, dan mesin berakhir di q_4 yang merupakan *final state*, maka **Diterima**.

Tapi kalau stringnya salah, misalnya aba : 1. Mulai q_0 , dapet 'a' → q_1 . 2. Di q_1 , dapet 'b' → q_2 . 3. Di q_2 , dapet 'a' → langsung dilempar ke *trap state* q_5 , karena polanya salah. String habis di q_5 (bukan *final state*), berarti **Ditolak**.

4.2 Kasus 2: Minimal Ada Satu Angka "1"

Kita pengen deteksi string yang penting ada angka "1"-nya. Bahasa rincinya $L(M) = \{w \in \{0, 1\}^* \mid w \text{ mengandung minimal satu } 1\}$.

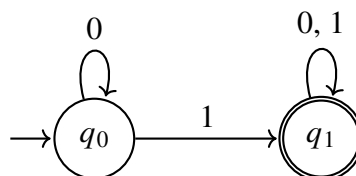


Figure 3: DFA Minimal Satu "1"

Konsepnya sederhana banget:

- **Fase q_0 (Pencarian):** Mesin nungguin angka 1. Kalau dapet 0 terus, mesin muter (loop) di situ aja dan statusnya tetep nunggu.
- **Fase q_1 (Ditemukan):** Sekali dapet angka 1, dia langsung pindah dan jadi *accept state*. Setelah itu, nggak peduli dikasih input 0 atau 1 berapa banyak pun, mesin tetep nahan posisinya di q_1 (stabilitas).

Contoh kalau dikasih string 0000, mesin cuma muter di q_0 empat kali, dan ujung-ujungnya **Ditolak**.

4.3 Kasus 3: Harus Diawali Angka "0"

Kita buat mesin buat himpunan $L = \{0, 00, 01, 000, 010, \dots\}$. Pokoknya wajib huruf pertama itu "0".

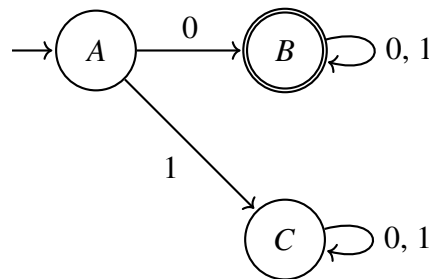


Figure 4: DFA Diawali "0". State C adalah Dead State.

Logikanya begini:

- Mulai di A. Cek langsung karakter pertama. Kalau "0", dia lolos masuk ke B (*Final State*). Di state B, huruf-huruf setelahnya (0 atau 1) udah nggak ngaruh lagi, dia bakal *loop* dan tetep diterima.
- Tapi, kalau karakter pertamanya "1", misalnya string 101, dari awal udah salah. Mesin langsung ngebuang ke C (*Dead State*). Dari sini, sisa angkanya (01) cuma diserap sama C dan akhirnya pasti ditolak.

4.4 Kasus 4: Panjang String Pas Banget 2 Karakter

Nah, kalau yang ini buat nyari string yang panjangnya persis dua karakter aja: $L = \{00, 01, 10, 11\}$. Perjalanannya gampang banget ditebak. Kalau masukin 001: - Karakter 1 ('0') pindah dari A

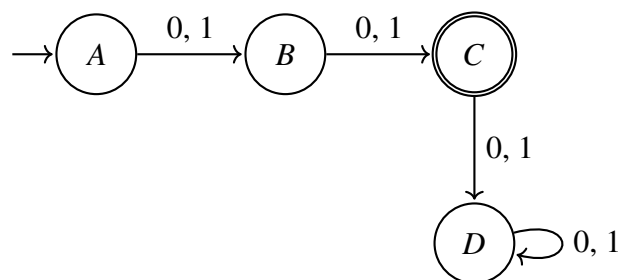


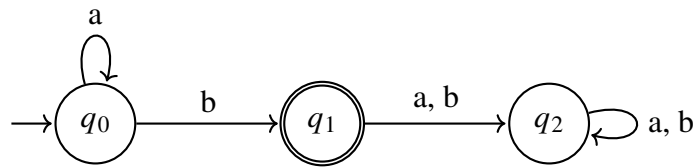
Figure 5: DFA Panjang Persis 2 Karakter

ke B. - Karakter 2 ('0') pindah dari B ke C (posisi saat ini *accept*). - Sayangnya ada karakter 3 ('1'). Mesin dilempar dari C ke *Dead State* D. Akhirnya **Ditolak**.

4.5 Kasus 5: Pengecekan Pola Terstruktur ($a^n b$)

Kita mau buat bahasa yang isinya cuma huruf 'a' berturut-turut (boleh nggak ada 'a' sama sekali, makanya $n \geq 0$), tapi wajib ditutup dengan tepat satu huruf 'b'. Jadi bahasa yang diterima itu $\{b, ab, aab, aaab, \dots\}$.

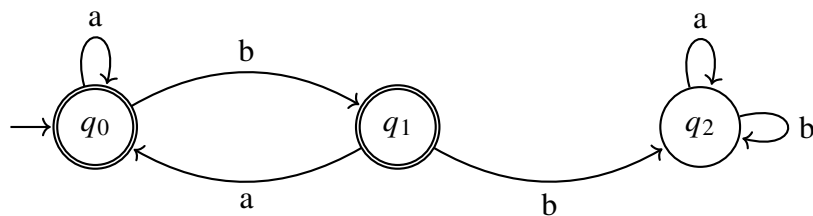
- Kalau masukin *aab*: Di q_0 mesin bakal baca 'a' dua kali (murni *looping*). Lalu baca 'b' dan pindah ke q_1 . Karena string habis dan q_1 itu *final*, mesin **Menerima** (*Accept*).

Figure 6: DFA Pola $a^n b$

- Kalau masukin *bab*: Awal baca 'b' mesin langsung pindah manis ke q_1 . Eh, pas huruf selanjutnya 'a' muncul, karena aturannya cuma boleh ada satu 'b' di akhir, mesin langsung ngebuang ke *trap state* q_2 . Akhirnya string *bab* **Ditolak**.

4.6 Kasus 6: Tracking Panjang Fungsi Transisi (δ^*)

Biar makin paham cara mesin mikir, kita simulasikan penelusuran string panjang pakai fungsi transisi diperluas. Diketahui DFA dengan $\Sigma = \{a, b\}$, $F = \{q_0, q_1\}$, dan aturannya:

Figure 7: DFA Multi-Final State. State q_2 bertindak sebagai Dead State.

Kita masukin string yang lumayan panjang: **abababaa**. Kita bedah perlahan dari kiri ke kanan:

1. Dari q_0 baca 'a', mesin tetep di q_0 . (Sisa string: *bababaa*)
2. Dari q_0 baca 'b', mesin loncat ke q_1 . (Sisa string: *ababaa*)
3. Dari q_1 baca 'a', mesin balik lagi ke q_0 . (Sisa string: *babaa*)
4. Dari q_0 baca 'b', mesin loncat ke q_1 . (Sisa string: *abaa*)
5. Dari q_1 baca 'a', mesin balik lagi ke q_0 . (Sisa string: *baa*)
6. Dari q_0 baca 'b', mesin loncat ke q_1 . (Sisa string: *aa*)
7. Dari q_1 baca 'a', mesin balik lagi ke q_0 . (Sisa string: *a*)
8. Dari q_0 baca 'a', mesin ngeloop di q_0 . (Sisa string habis)

Tracing atau penelusuran ini berakhir di state q_0 . Karena q_0 itu termasuk state AKHIR ($q_0 \in F$), maka kalimat *abababaa* sukses **Diterima**. Kalau kita perhatiin, mesin ini sebenarnya nerima pola selang-seling, asalkan dia nggak nemu "bb" berurutan yang bakal ngebuang dia ke q_2 .